It's not time to write the Ledger class yet. You're not testing its behavior here; you're testing the API class. Instead, you'll need something to stand in for a Ledger instance. Specifically, you'll need a *test double*.

Test Doubles: Mocks, Stubs, and Others

A test double is an object that stands in for another one during a test. Testers tend to refer to them as *mocks*, *stubs*, *fakes*, or *spies*, depending on how they are used. RSpec supports all of these uses under the umbrella term of *doubles*. We'll explain the differences in *Understanding Test Doubles*, or you can read Martin Fowler's article "Test Doubles" for a quick summary.[4]

To create a stand-in for an instance of a particular class, you'll use RSpec's instance_double method, and pass it the name of the class you're imitating (this class need not actually exist yet). Since you'll need to access this phony Ledger instance from all of your specs, you'll define it using a let construct, just like you did with the sandwich object in the first chapter.

There are a couple of other additions to make to your spec, which we've highlighted for you. Alter api_spec.rb to the following structure:

05-unit-specs/03/expense_tracker/spec/unit/app/api_spec.rb

```
require_relative '../../../app/api'
require 'rack/test'

module ExpenseTracker
  RecordResult = Struct.new(:success?, :expense_id, :error_message)

  RSpec.describe API do
    include Rack::Test::Methods

    def app
      API.new(ledger: ledger)
    end

    let(:ledger) { instance_double('ExpenseTracker::Ledger') }

    describe 'POST /expenses' do
      context 'when the expense is successfully recorded' do
        # ... specs go here ...
      end

      context 'when the expense fails validation' do
        # ... specs go here ...
      end
    end
  end
end
```

4. https://martinfowler.com/bliki/TestDouble.html